



2^{ème} Partie : Deep Learning

Chapitre 3 : Réseaux de neurones

ENSA Khouribga

Pr. A. DARGHAM

Master « BDAD » (2^{ème} Année)

Année académique : 2020/2021

Apprentissage supervisé

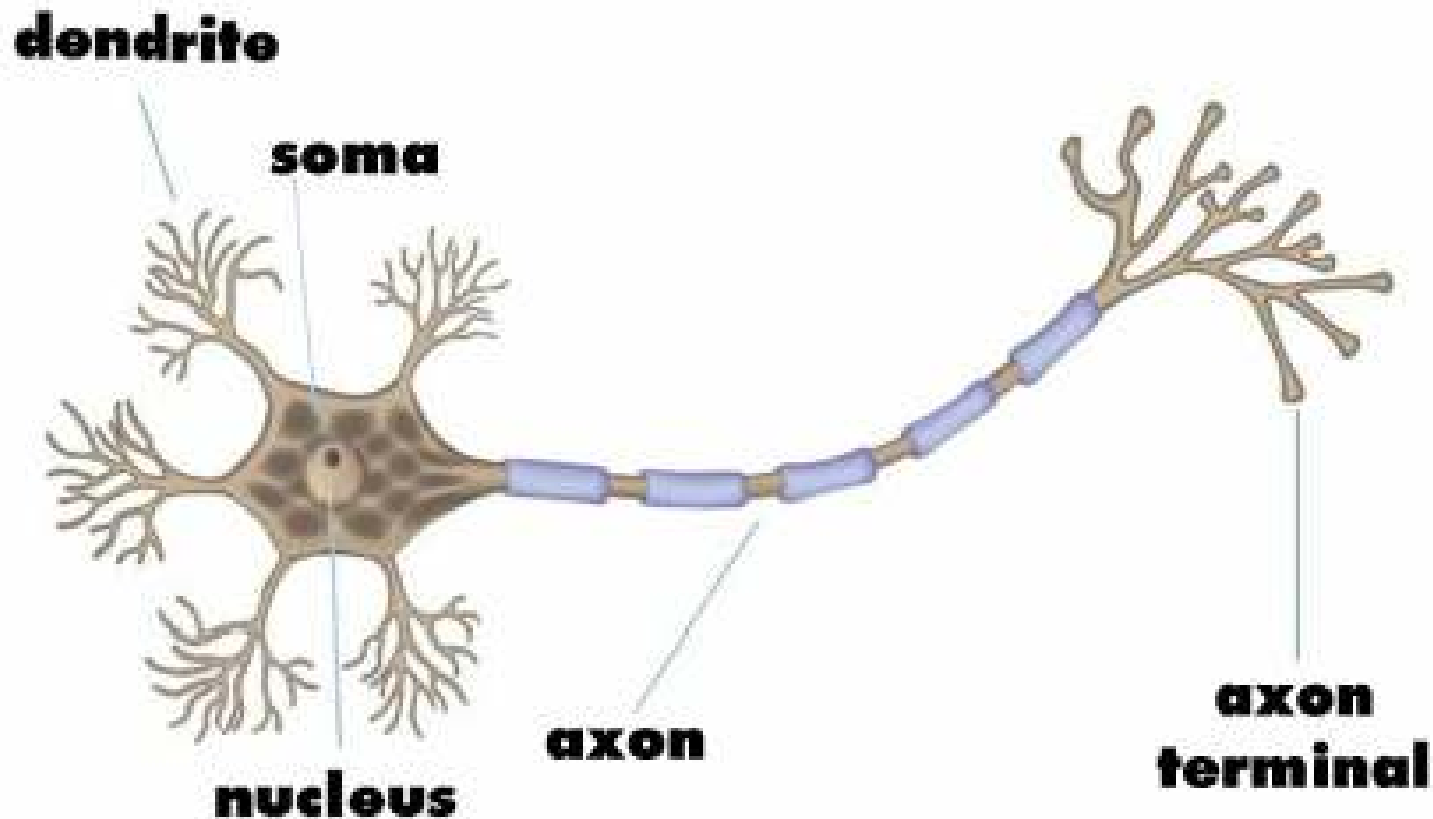
RÉSEAUX DE NEURONES

Réseaux de neurones

- **Introduction**

- Les **réseaux de neurones** sont inspirés par le **fonctionnement** du **cerveau humain**.
- Un cerveau humain peut traiter d'énormes quantités d'informations en utilisant des données envoyées par les sens humains (en particulier la vision).
- Le traitement est effectué par des **neurones**, qui travaillent sur les **signaux électriques** qui les traversent.

Réseaux de neurones



Structure d'un neurone biologique

Réseaux de neurones

- **Composantes principales d'un neurone biologique**
 - **Dendrites** : **points d'entrée** dans chaque **neurone** qui reçoivent des données des autres neurones du réseau sous forme **d'impulsions électriques**.
 - **Noyau** : elle génère des inférences à partir des entrées de dendrite et décide de l'action à entreprendre.
 - **Bornes axonales** : elles transmettent des sorties sous forme **d'impulsions électriques** au neurone suivant (**amplificateurs** de signaux).

Réseaux de neurones

- **Caractéristiques d'un neurone biologique**
 - Chaque **neurone** ne traite les signaux que s'ils dépassent un certain **seuil**.
 - Les **neurones** se déclenchent ou bien ne se déclenchent pas : ils ont donc un **comportement tout ou rien** (ou booléen 0/1).

Réseaux de neurones

- **Réseaux de neurones et IA**

- Les **réseaux de neurones artificiels (RNA)** au sein de l'**IA** existent depuis les années 1950, mais nous les avons rendus plus dominants au cours des dix dernières années en raison des progrès de l'informatique et des performances de calcul.
- Il y a eu des avancées majeures dans le traitement informatique, conduisant au **parallélisme massif**, les **représentations** et **calculs distribués**, la **capacité d'apprentissage** et de généralisation, la tolérance aux pannes et la faible consommation d'énergie.

Réseaux de neurones

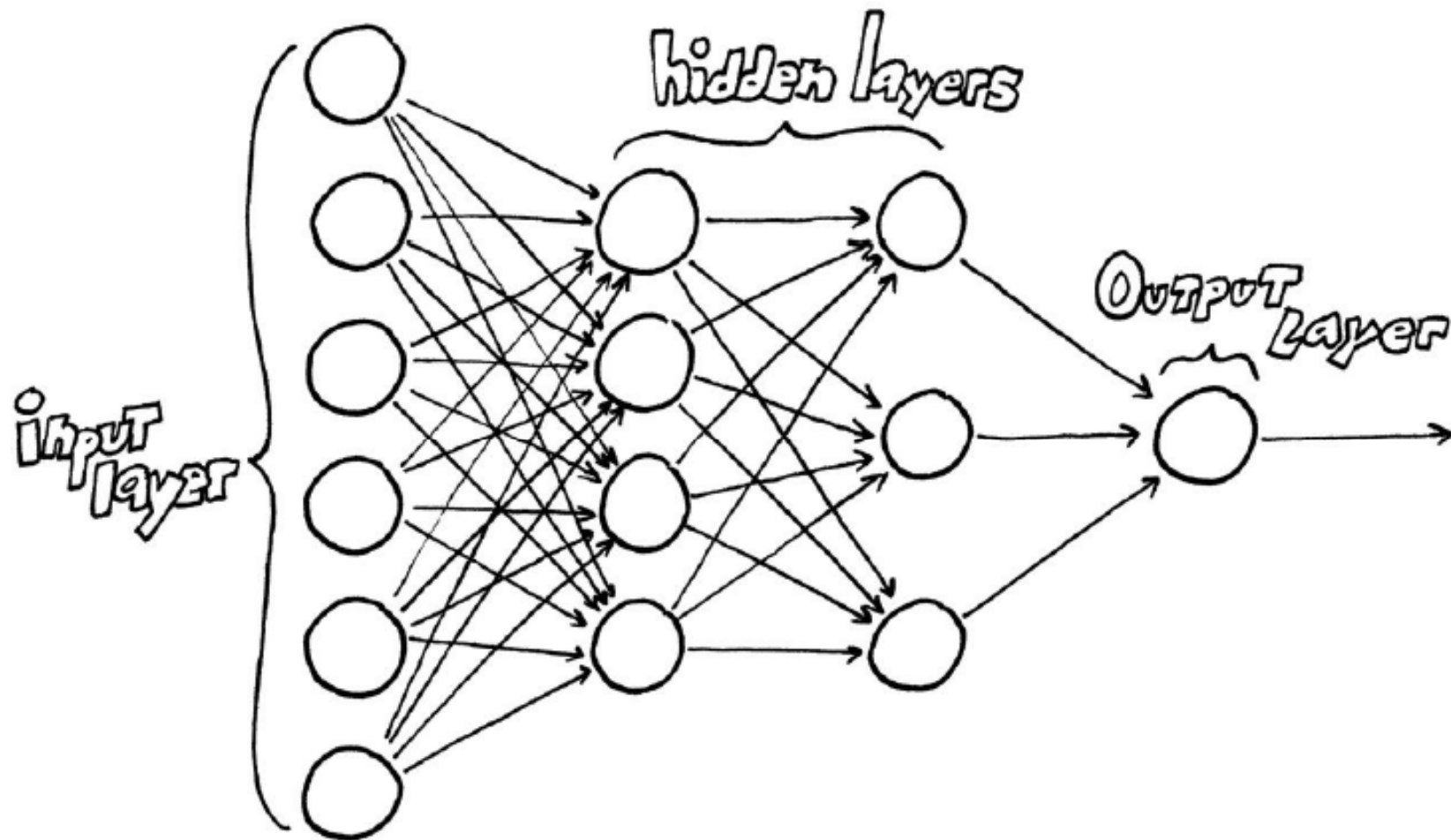
- **Domaines d'application des RNA**
 - Approximation de fonctions
 - Contrôle de processus
 - Reconnaissance/classification de formes
 - Clustering (Regroupement)
 - Systèmes de prédiction
 - Optimisation des systèmes

Réseaux de neurones

- **Fonctionnement des RNA**

- Similaire à une structure de neurones biologiques, les **RNA** définissent le **neurone artificiel** comme une **unité centrale de traitement**, qui effectue une **opération mathématique** pour générer une **sortie** à partir d'un **ensemble d'entrées**.
- La **sortie d'un neurone** est **fonction** de la **somme pondérée des entrées** et du **biais**.
- Chaque **neurone** effectue une opération très simple qui **implique l'activation** si la quantité totale de signal reçue dépasse un **seuil d'activation**.

Réseaux de neurones



Structure d'un RNA

Réseaux de neurones

- **Fonctionnement des RNA**

- La tâche fondamentale d'un **RNA** est tout simplement le **calcul des sorties** de tous les **neurones** (il s'agit d'un calcul entièrement **déterministe**).
- Pour l'essentiel, un **RNA** est **l'approximation** d'une certaine **fonction mathématique**.
- Nous allons maintenant introduire une nouvelle terminologie associée aux **RNA** : la **couche d'entrée**, les **couches cachées**, la **couche de sortie**, les **poids**, le **biais** et les **fonctions d'activation**.

Réseaux de neurones

- **Fonctionnement des RNA**

- Dans un **RNA**, on trouve :

- Un **ensemble d'entrées** : elles forment la **couche d'entrée** → représentée par un **vecteur** x .
 - Un **processeur** : il est formé d'un **ensemble de neurones cachés** qui constituent les **couches intermédiaires** de calcul.
 - Un **ensemble de sorties** : elles forment la **couche de sortie**.

- La tâche d'un **RNA** est claire : transformer des **informations en entrée** en une **information en sortie**.

Réseaux de neurones

- **Fonctionnement des RNA**

- Les **poids** dans un **RNA** sont le **facteur le plus important** dans la conversion d'une entrée en une sortie.
- Les **poids** sont des **paramètres numériques** qui déterminent dans quelle mesure chacun des **neurones** affecte l'autre.
- Pour un **neurone typique**, si les **entrées** sont respectivement x_1 , x_2 et x_3 , alors les **poids synaptiques** à leur appliquer sont désignés respectivement par w_1 , w_2 et w_3 .

Réseaux de neurones

- **Fonctionnement des RNA**

- La **sortie** est :

$$y = f(x) = \sum_i w_i x_i$$

- Le **biais** est comme **l'interception** ajoutée dans une équation linéaire.

- Il s'agit d'un **paramètre supplémentaire** qui est utilisé pour **ajuster la sortie**.

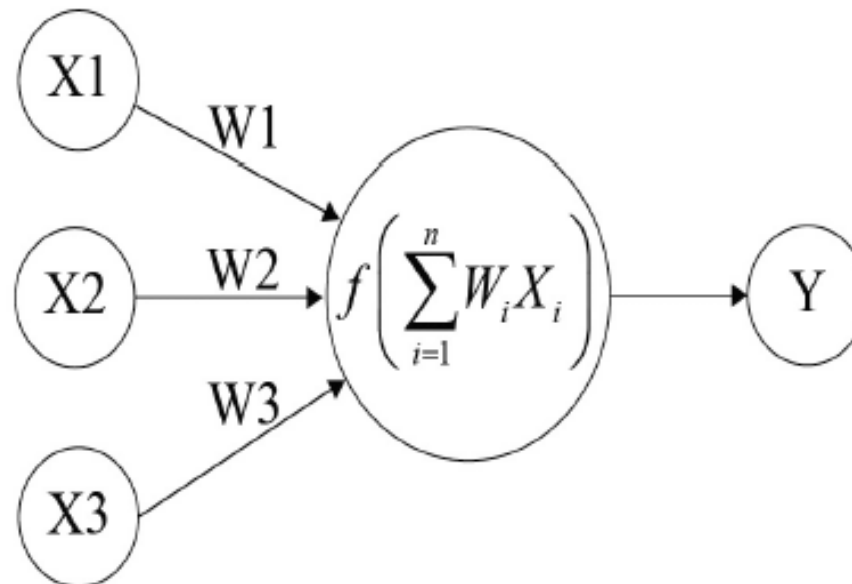
- Le traitement effectué par un **neurone** est ainsi noté :

$$\textit{Sortie} := \sum (\textit{poids} * \textit{entrées}) + \textit{biais}$$

Réseaux de neurones

- **Fonctionnement des RNA**

- Une **fonction d'activation** est une **fonction** appliquée sur cette **sortie**.
- **L'entrée** de la **couche suivante** est la **sortie des neurones de la couche précédente**, comme le montre la figure suivante :



Réseaux de neurones

- **Apprentissage d'un RNA**

- L'**apprentissage** d'un **RNA** consiste à présenter au réseau quelques exemples de données et à modifier les **poids** pour mieux se rapprocher de la **fonction** souhaitée.
- Une **période** est une **itération du processus** de l'introduction au réseau d'une entrée et de mise à jour des **poids** du réseau.
- Il s'agit d'une série complète de **rétroaction** et de **rétro-propagation** pour la mise à jour des poids.
- Il s'agit également d'une **lecture complète de l'ensemble de données**.

Réseaux de neurones

- **Fonctions d'activation**

- L'abstraction du traitement des **RNA** se fait principalement grâce aux **fonctions d'activation**.
- Une **fonction d'activation** est une **fonction mathématique** qui convertit l'entrée en une sortie et ajoute la "magie du traitement" des **RNA**.
- Sans fonctions d'activation, le fonctionnement des **RNA** sera comme des **fonctions linéaires**.
- Une **fonction linéaire** est une fonction où la sortie est proportionnelle à l'entrée, par exemple :

$$y = 2x + 1$$

Réseaux de neurones

- **Fonctions d'activation**

- Cependant, la plupart des problèmes que les **RNA** essaient de résoudre sont de nature non linéaire et complexe.
- Pour atteindre la non-linéarité, les fonctions d'activation sont utilisées.
- Exemples de fonctions non-linéaires :

$$y = x^2$$

$$y = \sin(x)$$

Réseaux de neurones

- **Exemples de fonctions d'activation**

- **Fonction linéaire identité :**

$$y = f(x) = x$$

- **Fonction signe** (utile pour la **classification binaire**) :

$$y = f(x) = 0, \text{ si } x < 0, \text{ et } f(x) = 1, \text{ si } x \geq 0$$

- **Fonction sigmoïde :**

$$y = f(x) = 1 / (1 + e^{-x})$$

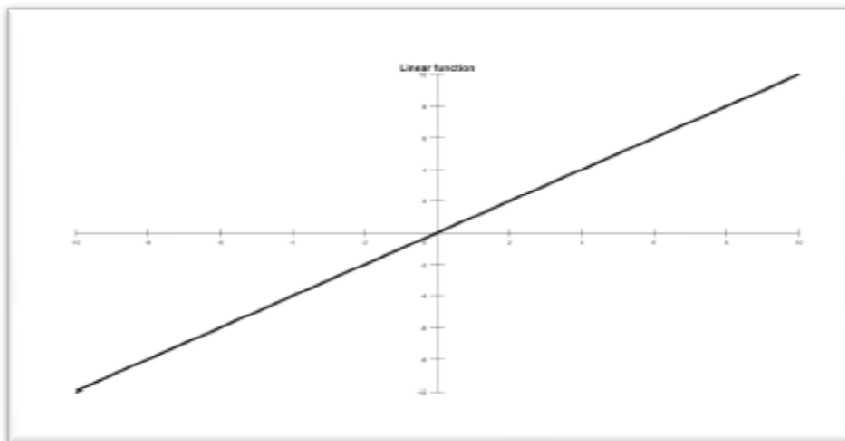
- **Fonction tangente hyperbolique :**

$$y = f(x) = \tanh(x)$$

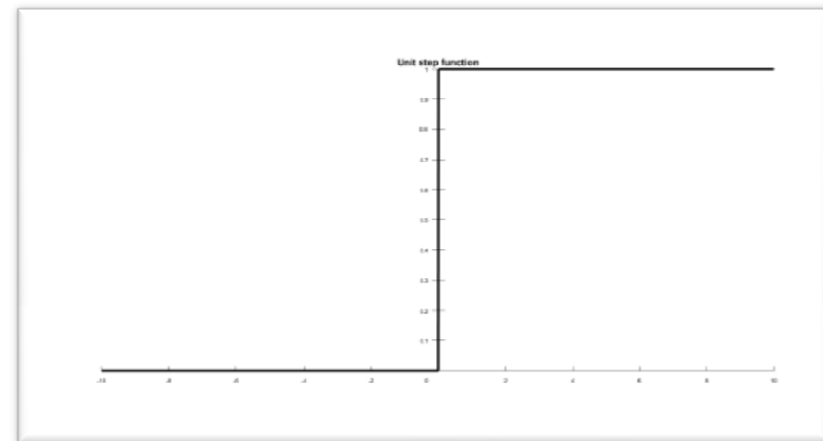
- **Fonction :**

$$y = f(x) = 0, \text{ si } x < 0, \text{ et } f(x) = x, \text{ si } x \geq 0$$

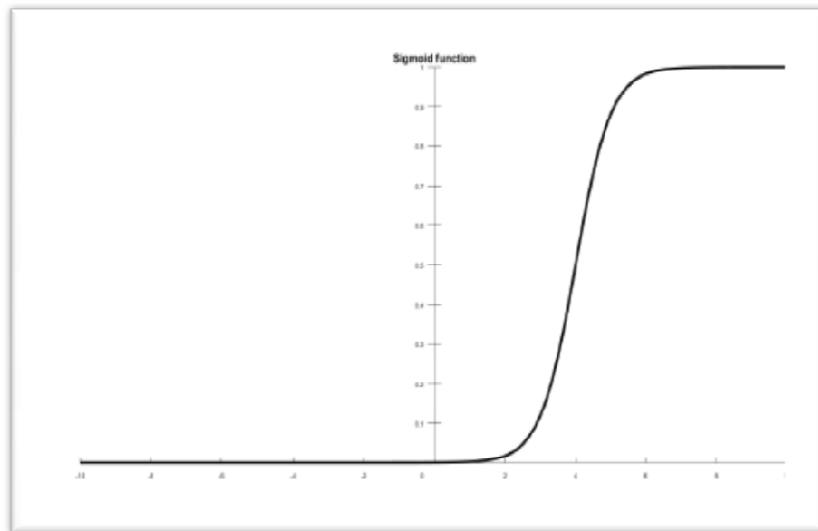
Réseaux de neurones



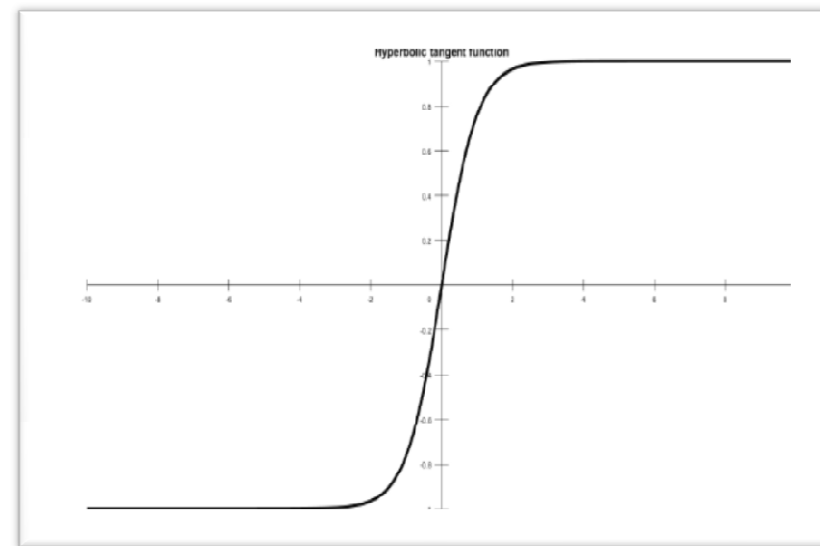
$$f(x) = x$$



$$f(x) = \text{signe}(x)$$

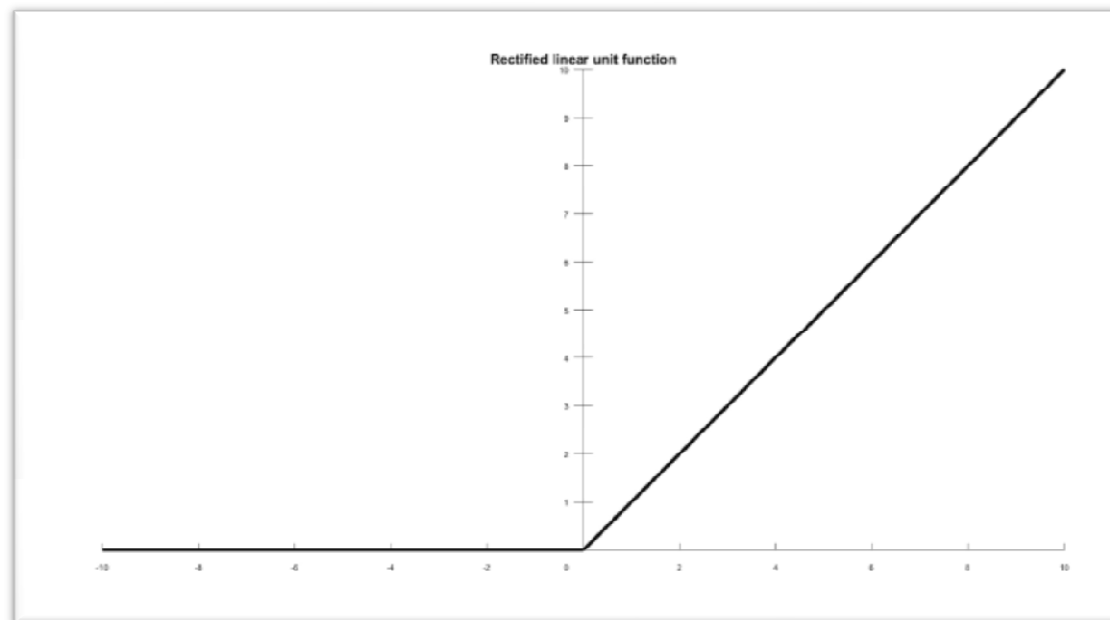


$$f(x) = 1/(1+e^{-x})$$



$$f(x) = \tanh(x)$$

Réseaux de neurones



$$f(x) = 0, \text{ si } x < 0 \text{ et } f(x) = x, \text{ si } x \geq 0$$

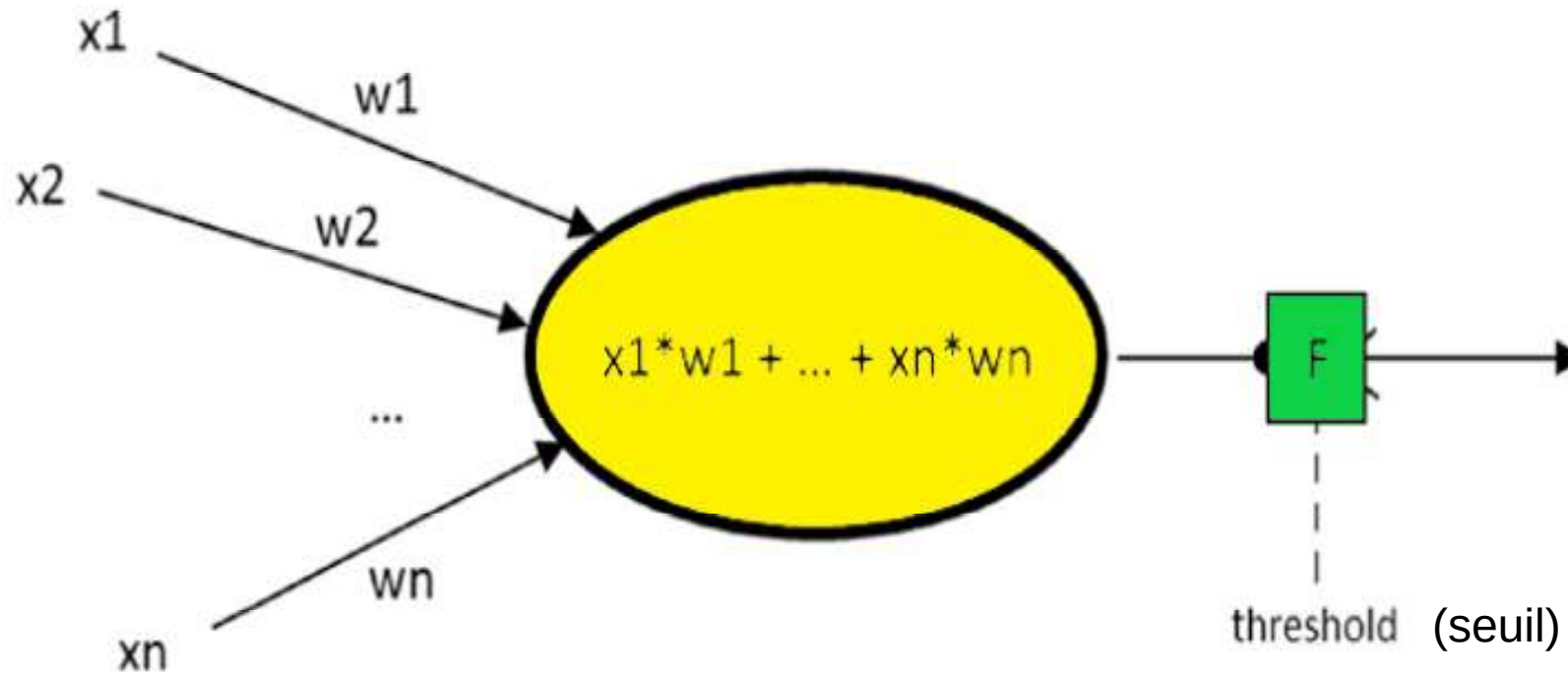
Réseaux de neurones

- **Le perceptron**

- Le **perceptron** est le modèle le plus simple d'un RNA.
- Un **perceptron** reçoit n entrées : $x_1, x_2, \dots, x_n$ (qui est un vecteur x). Il calcule ensuite, la **somme pondérée des entrées** (combinaison linéaire) : $w_1 * x_1 + \dots + w_n * x_n$.
- Il applique finalement une **fonction d'activation** f sur cette somme calculée pour déduire la sortie (1 ou 0) en prenant compte d'un **seuil** θ .
- Dans le cas d'un **perceptron**, la **fonction d'activation** est toujours de cette forme : $f(x) = 1$, si $x \geq \theta$; 0 sinon.

Réseaux de neurones

- Schéma d'un perceptron



Réseaux de neurones

- **Le perceptron**

- Le **mode** de **fonctionnement** d'un **perceptron** est le simple possible des **RNA**.
- Dans le cadre de **l'apprentissage supervisé**, notre objectif est la **classification**.
- Maintenant, comment pouvons-nous **entraîner** un **perceptron** à prédire correctement les nouvelles données ?
- Tout d'abord, le perceptron est un **classificateur linéaire** : il essaie de trouver un **vecteur de poids** et une **valeur de seuil** de telle manière que l'espace du problème soit divisé en deux classes A et B.

Réseaux de neurones

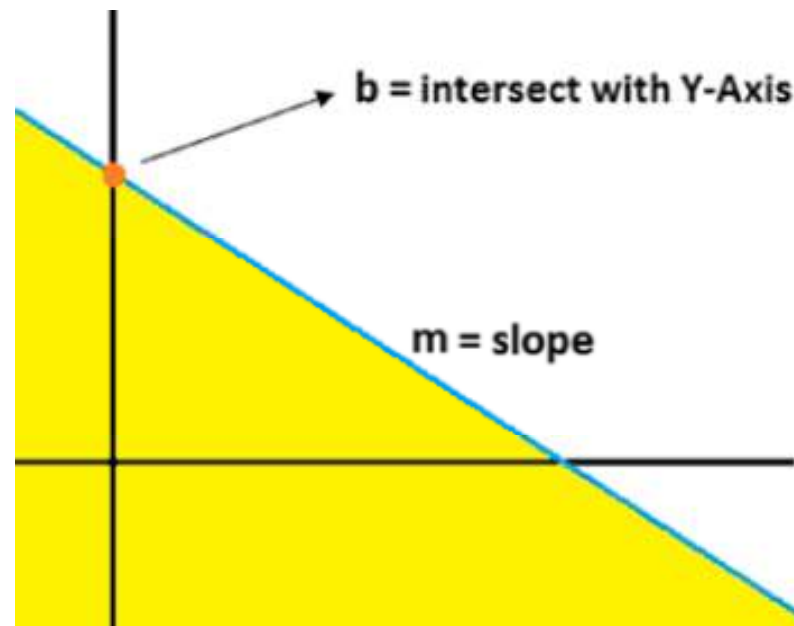
- **Le perceptron**

- Le **vecteur de poids** w et la **valeur de seuil** définiront l'un des **hyperplans de classification** possibles.
- Dans un espace 2D, l'hyperplan est une **droite**, alors que dans un espace 3D, il s'agit d'un **plan**.
- Il se produira toujours que tous les points des données d'apprentissage se trouveront d'un côté ou de l'autre de l'hyperplan de classification si l'ensemble de données d'apprentissage est linéairement séparable.

Réseaux de neurones

- **Exemple de perceptron en 2D**

- Considérons l'équation d'une droite : $y = mx + b$, où m est la **pente** de la droite (ou le **gradient**) et b est le **biais** déterminant le décalage, à gauche ou à droite (c'est l'intersection de la droite avec l'axe des y).



Réseaux de neurones

- **Le processus d'apprentissage du perceptron**
 - Le processus d'apprentissage dans un perceptron consistera en un **ajustement**, effectué au fil du temps à travers plusieurs itérations, aux **poids** (pente de la droite) et aussi au **biais** pour tenter de trouver une droite (**hyperplan**) qui classe correctement toutes les données d'entraînement.

Réseaux de neurones

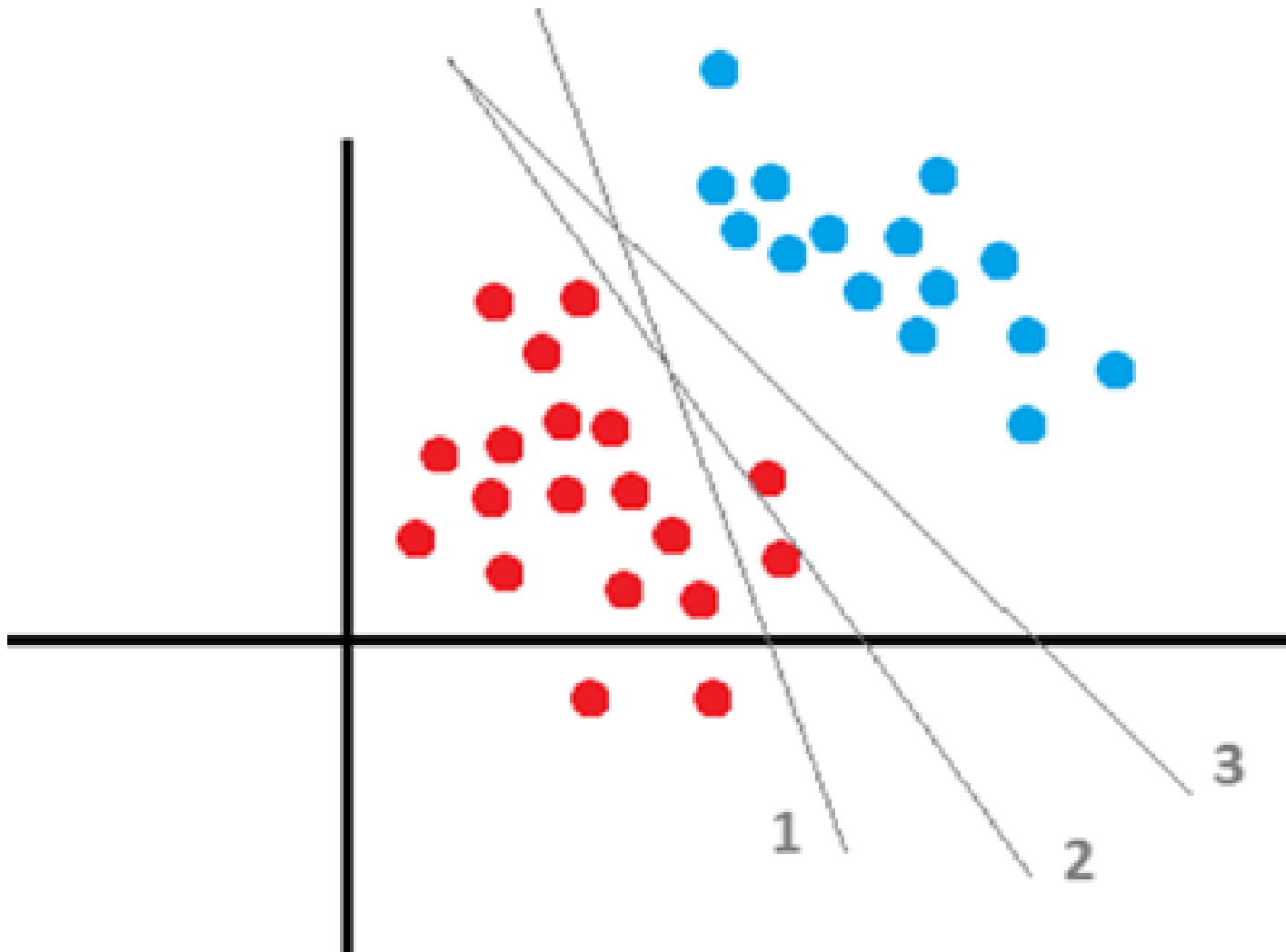
Algorithme du perceptron

- L'algorithme commence par définir des **valeurs aléatoires** pour le **vecteur de poids** w (généralement dans l'intervalle $[0, 1]$), ainsi que pour le **biais** $b \Rightarrow$ construction d'un **hyperplan de classification aléatoire**.
- Ensuite, afin d'**affiner l'hyperplan de classification** et de le forcer à classer correctement tous les exemples, une boucle à travers les **exemples d'apprentissage** est effectuée et chaque **erreur détectée dans la classification** d'une seule donnée d'entraînement est **corrigée** en augmentant ou en diminuant le **poids** associé à la composante de cette donnée.
- Ensuite, le **biais** est également corrigé.

Réseaux de neurones

- **Le processus d'apprentissage du perceptron**
 - Le **processus d'apprentissage d'un perceptron** est essentiellement une **technique d'optimisation** dans laquelle on **améliore l'hyperplan de classification** en modifiant lentement sa pente (son **gradient**) et le **biais** pour le déplacer vers une position où toutes les données d'entraînement seraient correctement classées.

Réseaux de neurones



Réseaux de neurones

Pseudo-code d'un perceptron

- Initialiser aléatoirement les **poids** w_i et le **bias** b .
- Fixer le **taux d'apprentissage** α dans l'intervalle $]0, 1[$.
/* La valeur de α est typiquement 0.05 */
- Tant que la **condition d'arrêt n'est pas satisfaite**, Répéter :
 - ➊ Prendre un **exemple d'apprentissage** (x, y) dans le dataset S .
 - ➋ Calculer la sortie y_x du perceptron pour l'exemple x .
 - ➌ Si $(y \neq y_x)$, alors mettre à jour les poids et les biais :

$$w_i = w_i + \alpha \cdot (y - y_x) \cdot x_i$$

$$b = b + \alpha \cdot (y - y_x)$$

- Retourner le **vecteur des poids** w et le **biais** b .